



Starting Agentic SDLC with GitHub Copilot

GitHub Copilot's interaction model is expanding from completing code to executing bounded modernization work under human governance.

Shinya Yanagihara

Sr Developer Global Black Belt Specialist — Asia Pacific, Microsoft



GitHub Copilot's next value is not generating more code; it is enabling teams to redesign modernization as bounded, verifiable work that agents execute and humans govern.

- **Completion** assists the next implementation move
- **Delegation** assigns a bounded, verifiable outcome
- **Modernization** makes the shift concrete



The Agent-Native Engineering System

Humans set direction. Agents close the loop.

What if three modernization PRs were waiting

3

Imagine starting the week with three review-ready modernization PRs—each scoped, evidenced, and still waiting for human approval.

- Dependency-analysis PR maps components, constraints, and risks
- Characterization-test PR captures current behavior
- Runtime-upgrade PR includes changes and validation evidence
- Nothing auto-merges; humans review sequence, evidence, and risk

The overnight value is prepared evidence, not an automatic merge.



We learned to ask AI for code. Are we ready to delegate work?

The interaction changes when we stop requesting an artifact and start defining accountable work.

Requesting output

- Generate → correct → repeat optimizes a local coding turn
- Define the outcome, context, and boundaries

Delegating accountable work

- Require checks and reviewable evidence
- Approve, revise, or reject the result

A prompt asks for output; delegation defines accountable work.



Three things to take home

GitHub Copilot's next value is not generating more code; it is enabling teams to redesign modernization as bounded, verifiable work that agents execute and humans govern.

1

Lens

distinguish completion,
collaboration, delegation, and
orchestration

2

Delegation Contract

outcome, context, boundaries,
checks, evidence

3

First action

choose one contained
modernization outcome

Leave with a way to see, specify, and start agentic work.



Modernization was never just code conversion

Modernization is a system change whose code edits sit inside discovery, decisions, validation, rollout, and operations.

- 1 Recover requirements, rules, dependencies, and current behavior
- 2 Choose the target state and sequence migration waves
- 3 Change code and tests without losing intended behavior
- 4 Validate security, rollout readiness, and operational fit
- 5 Observe production and feed learning into the next wave

Code conversion is one workstream inside a governed system change.



Legacy has three forms

Modernizing code alone leaves the knowledge and process that constrain change untouched.

Code legacy:

obsolete APIs, frameworks, tests, and build paths

Knowledge legacy:

tacit rules, missing rationale, unclear ownership

Process legacy:

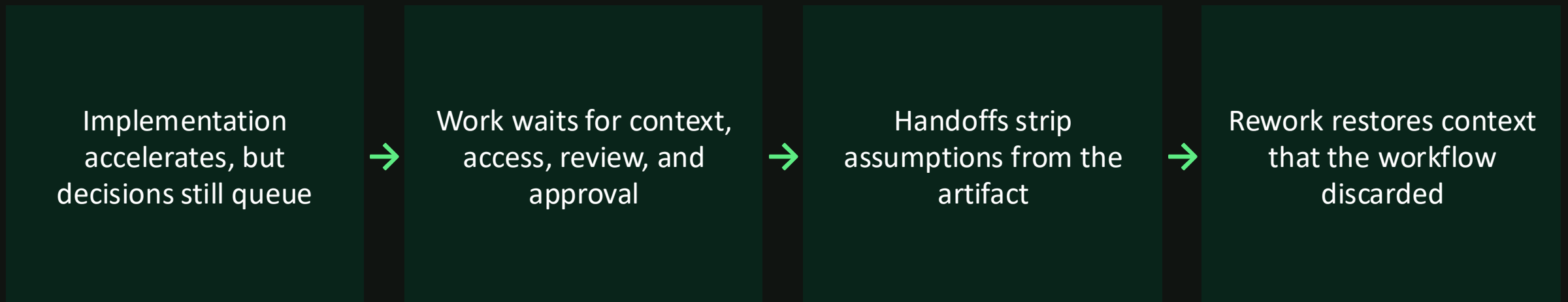
handoffs, approvals, queues, and release habits

The hardest legacy may live outside the repository.



The modernization paradox

Faster coding does not eliminate the waiting and context loss surrounding the code.



Optimizing coding alone can make waiting more visible, not less.



Automation follows a path. Delegation pursues an outcome.

Automation fits known paths; delegation fits clear outcomes whose execution requires bounded contextual choices.

Automation follows a path

Automation: known inputs follow a predetermined path

Delegation pursues an outcome

Delegation: a bounded outcome permits contextual choices

SHARED VERIFICATION BACKBONE

- Agents invoke existing scripts, tests, linters, and CI
- Deterministic checks remain the verification backbone

Keep deterministic controls; delegate the work around them.



The opportunity is to redesign the handoffs

Each handoff should preserve intent, ownership, provenance, and evidence instead of passing an ambiguous artifact.



Code → Doc: implementation becomes durable context

Doc → Spec: intent becomes bounded requirements

Spec → Backlog: requirements become reviewable work

Work → PR: execution returns changes plus evidence

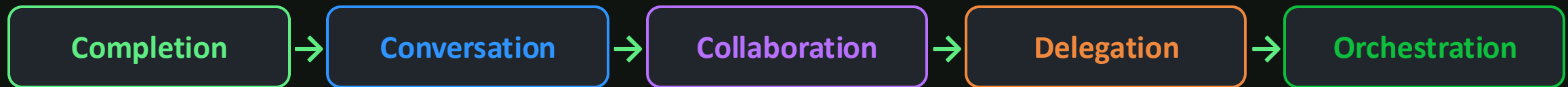
Telemetry → Issue: runtime signals become owned follow-up

Handoffs become traceable work products.



Copilot evolved by increasing the unit of delegation

The useful progression is measured by how much work a person can define and hand off—not by launch dates.



Completion: token, line, or block

Conversation: question, explanation, or analysis

Collaboration: bounded steps in a shared session

Delegation: outcome returned with evidence

Orchestration: repeatable flow across governed work

The unit of work grows; human accountability remains.



Delegation transfers a bounded outcome

Delegation transfers responsibility for executing an outcome, not authority to merge it.

Completion	Conversation	Collaboration	Delegation	Orchestration
DELEGATED UNIT • Outcome: upgrade the runtime while preserving the public API • Boundary: target repository and approved dependency range		COPILOT / AGENT WORK • Context: API contract and current architecture • Checks: characterization tests plus required validation		
HUMAN ROLE • <i>Retains the approval and merge decision</i>		RETURN PACKAGE • Return: draft PR, evidence, and unresolved incompatibilities 4 / 4		
A useful delegation returns evidence and explicit uncertainty.				



Orchestration delegates a repeatable flow

Orchestration coordinates bounded work under policy, permissions, deterministic checks, and human review gates.

Repeatability

- Assess dependencies and sequence modernization work
- Update the plan from verified outcomes

Coordination

- Create issues and assign low-risk, bounded tasks
- Generate tests and review returned pull requests

Governed flow

- Escalate ambiguity, failed checks, or high-risk changes

Orchestration is policy-bound coordination—not unrestricted autonomy.



Copilot and Agent are not opposites

Copilot and agent describe interaction levels within GitHub Copilot, not rival products.

Interaction level	Management object / human relationship
Completion	Completion accelerates implementation while the developer owns the task
Conversation	Conversation delegates analysis while the developer acts on the answer
Collaboration	Collaboration shares bounded multi-step work
Delegation	Delegation transfers a bounded outcome
Orchestration	Orchestration coordinates a repeatable governed flow

Completion → Conversation → Collaboration → Delegation → Orchestration



The management object changes

As delegation grows, teams manage work definitions, outcomes, and evidence—not only prompts and generated snippets.

Prompt-centric management

Prompt quality

Generated code

Local session



Work-system management

work definition

outcome and evidence

governed system throughput

Prompt engineering becomes work engineering.



The new developer skill is delegation design

Developers increasingly design the conditions under which agent work can be trusted, reviewed, and reused.

- 1 **Decompose** outcomes into bounded, independently verifiable work
- 2 **Define** acceptance criteria, context, permissions, and constraints
- 3 **Attach** deterministic checks and required evidence
- 4 **Review** evidence; resolve ambiguity; approve or reject
- 5 **Decide** what not to delegate; capture reusable knowledge

Good delegation makes agent work bounded, observable, and reviewable.



What is Agentic SDLC?

Agentic SDLC is a governed software development lifecycle in which agents pursue explicit outcomes within bounded scope, use approved context and tools, and return verifiable evidence for human decisions.

- **Bounded** — Explicit outcomes, scope, permissions, and stopping conditions
- **Contextual** — Relevant, current, and authorized project knowledge
- **Tool-using** — Approved tools for reading, changing, running, and observing
- **Verifiable** — Diffs, checks, outputs, assumptions, and exceptions
- **Governed** — Humans retain risk-bearing decisions and accountability

GitHub Copilot's next value is not generating more code; it is enabling teams to redesign modernization as bounded, verifiable work that agents execute and humans govern.



Agentic does not mean autonomous everything

Agentic delivery delegates preparation and execution without transferring accountability. Humans retain decisions that carry architectural, operational, or business risk.

Agents execute

- Agents investigate, draft plans, and prepare bounded changes
- Agents execute approved work using allowed tools
- Agents run checks and report evidence, uncertainty, and exceptions

Humans decide

- Humans set priorities, architecture, constraints, and acceptable risk
- Humans redirect work and decide approval, release, and production actions

Delegate execution; retain judgment and accountability.



Five layers make delegation operational

Delegation becomes operational when intent, context, agency, execution, and governance work as one system.

Intent — Desired outcome, rationale, boundaries, and success conditions

Context — Persistent knowledge, task inputs, and current observations

Agent — Planning and action within the delegation contract

Execution — Approved tools, environments, actions, and feedback

Governance — Evidence, escalation, approvals, and accountability

HUMAN ACCOUNTABILITY

The human decision maker crosses all five layers.



Context is not a bigger prompt

Context is a managed system of relevant, current, and authorized information—not a one-time prompt enlarged until it works.

Persistent context

Architecture, conventions, policies, and repository knowledge

Task context

Outcome, scope, acceptance criteria, and supplied references

Dynamic context

Current code, tool results, failures, and changed assumptions

CONTEXT-QUALITY REQUIREMENTS

- Context must be discoverable where agents can reliably find it
- Context must be permission-aware, owned, reviewed, and maintained

More tokens do not repair stale, hidden, or unauthorized context.



Safe delegation needs a contract

A Delegation Contract makes success, boundaries, proof, and decision gates inspectable before work begins.

Outcome + Why

Observable result and its business or technical purpose

Scope + Out

Where work may act and what explicitly must not change

Context

Repository knowledge, current behavior, decisions, and assumptions

Tools + Constraints

Permitted capabilities, limits, policies, and budgets

Acceptance

Observable conditions that define done

Verification + Evidence

Repeatable checks and proof returned for review

Escalate

Ambiguity, risk, or failure conditions that stop the work

Human gates

Named decisions that remain with accountable people

No delegation is complete without an observable outcome, evidence, and a human gate.



Autonomy is a budget, not a switch

Autonomy has multiple dimensions that teams allocate per task. Expand it only when work is reversible, contained, and strongly evidenced.

Scope budget — Files, components, repositories, and issue boundaries

Capability budget — Tools, network, and PR or workflow permissions

Compute budget — Runtime and parallelism

Decision budget — Required approvals and stopping conditions

SCALE EVERY BUDGET

- Scale every budget to reversibility, blast radius, and evidence quality

Increase autonomy one bounded dimension at a time.



Reason probabilistically. Verify deterministically.

Use agent reasoning to interpret, explore, and propose. Accept work through repeatable checks with explicit pass or fail criteria.

Probabilistic reasoning

- Agent reasoning interprets intent, locates patterns, and proposes options
- Builds and tests validate compilation and specified behavior
- Static analysis and CodeQL evaluate configured rules and queries

Deterministic verification

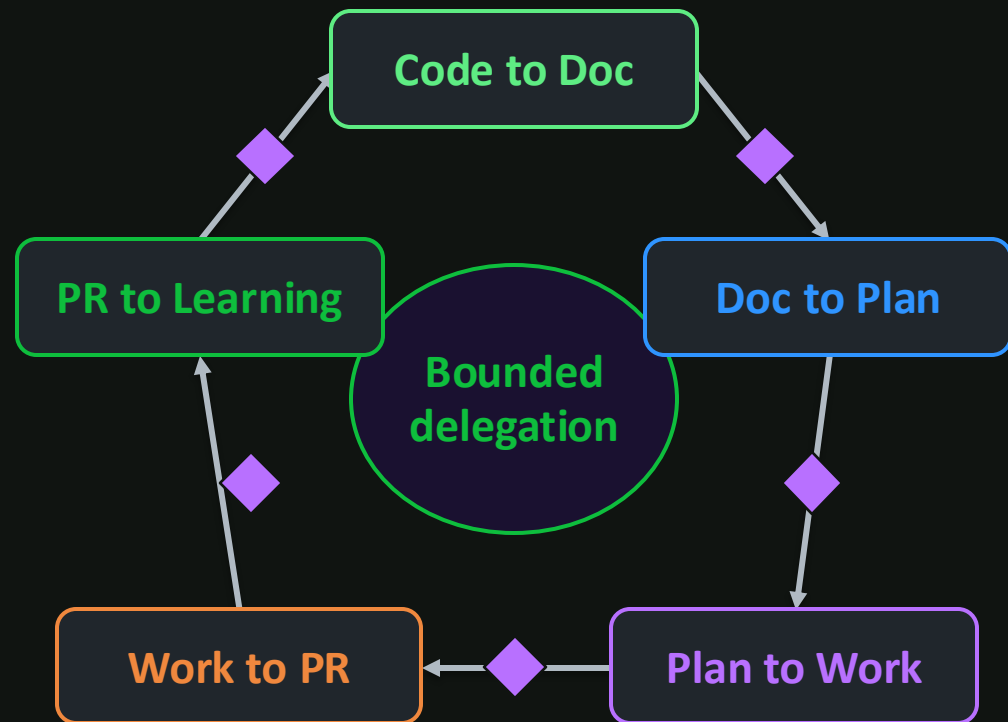
- Dependency and policy checks enforce approved constraints
- Performance checks compare controlled runs with defined thresholds

Reason probabilistically. Verify deterministically.



The Agentic Modernization Loop

The loop turns uncertainty into small, reviewable changes and feeds every reviewed result back into organizational knowledge.



◆ Human gate

Code to Doc :

Recover structure, behavior, rules, and uncertainty

Doc to Plan :

Define the target, options, risks, and migration waves

Plan to Work :

Create bounded, independently verifiable tasks

Work to PR :

Implement, check, report, and request human review

PR to Learning :

Improve instructions, tests, plans, and future work

Bounded delegation sits at the center; human gates connect every stage.



Running scenario: a 15-year-old customer management system

This scenario requires modernization without sacrificing known behavior or service availability.

Legacy system

- A 15-year-old Java customer-management system

Modernization constraints

- Outdated framework and dependencies
- Sparse documentation and limited automated tests
- Business rules distributed across code and operational knowledge

Delivery rule

- Incremental migration only: no big-bang rewrite, with availability maintained

Modernize the system without losing its behavior, knowledge, or availability.



Recover intent with Code to Doc

The first modernization deliverable is not changed code. It is a shared, reviewable account of the current system.

Structure

- Inventory modules, entry points, and dependencies
- Trace critical user, data, and integration flows

Behavior and rules

- Extract business rules from code and operational knowledge
- Produce concise system and flow diagrams; flag contradictions

Uncertainty

- Record assumptions and unknowns for human validation

Recovered documentation is a hypothesis until domain owners validate it.



Create a behavioral safety net

Before changing implementation, capture enough current behavior to detect unintended change while leaving intent decisions to humans.

- 1 Prioritize critical paths and consequential failure modes
- 2 Add characterization tests for observable current behavior
- 3 Use representative data, boundaries, and failure cases
- 4 Identify untestable areas and strengthen observability where possible

HUMAN DECISION

- Humans decide which behavior is intended and which is a defect

Protect known behavior before changing its implementation.



Assess opportunities with evidence

Assessment converts modernization intent into an evidence-backed opportunity map. It does not choose the strategy by itself.

- | | |
|---|--|
| 1 | Runtime and framework versions, support status, and unsupported dependencies |
| 2 | Known vulnerabilities and relevant security findings |
| 3 | Coupling and likely change boundaries |
| 4 | Build reproducibility and test coverage gaps |
| 5 | Deployment friction plus operational and cloud constraints |

FACTUAL ASSESSMENT — NOT THE STRATEGY DECISION

- Humans choose the modernization strategy from this evidence

Prioritize modernization by evidence, risk, and feasibility—not novelty.



Define the target and migration strategy

Compare options before implementation, then choose a staged path that humans can verify, stop, or roll back.

Target state

- Describe the as-is and to-be states in behavioral and technical terms
- Compare migration options and trade-offs

Migration sequence

- Map migration dependencies and sequence them into waves
- State risks, prerequisites, and rollback for every wave

Reversibility and acceptance

- Define acceptance evidence; humans choose strategy, sequence, and risk

No big-bang rewrite: every wave must be independently verifiable and reversible.



Turn the plan into delegable work

A delegable issue is small enough to contain and complete enough to verify.

Bad issue

- “Modernize the customer service.”

Good issue

- “Upgrade customer-api validation; preserve behavior; pass checks; revert one PR to roll back.”

DELEGATION CRITERIA — APPLY TO BOTH

- Define one observable outcome, explicit dependencies, and clear boundaries
- Limit blast radius to one component or coherent change set
- Name available verification and a practical rollback

Make work small enough to review and complete enough to prove.



Delegate implementation and retain judgment

The agent executes a bounded workflow and returns a draft PR with evidence. Humans can redirect the work and retain approval authority.

● Agent

● Human

- 1 Approved issue → **agent** research → proposed plan
- 2 **Human** confirms or redirects scope, approach, and assumptions
- 3 **Agent** works on a branch within the approved boundaries
- 4 **Agent** runs checks and reports results, diffs, and exceptions
- 5 Agent opens a draft PR; **humans** review, request changes, approve, or reject

The PR is a collaboration surface, not a transfer of accountability.



Review the outcome, not its origin

Evaluate AI-produced work against the same engineering obligations as other changes, while requiring evidence proportional to uncertainty and impact.

- | | |
|---|--|
| 1 | Does the result satisfy requirements and acceptance criteria? |
| 2 | Does it preserve architecture and compatibility constraints? |
| 3 | Does it meet security, dependency, and policy requirements? |
| 4 | Do tests and operational evidence support the change? |
| 5 | Is the change reversible, and are documentation and decisions updated? |

AI-produced work warrants stronger evidence, not automatic distrust.



From “modernize this” to a Delegation Contract

A Delegation Contract converts an ambiguous modernization request into bounded authority, observable acceptance, and explicit human gates.

Outcome / Why Move the auth module to supported APIs and reduce unsupported security risk	Scope / Out Change auth code, configuration, tests, and required dependencies; exclude roles, UI, schema, and rollout	Context Current login, session, and API behavior; dependency baseline; characterization tests; approved repository guidance	Tools / Constraints Use repository and CI tools with approved dependencies; preserve behavior; expose no production secrets
Acceptance Supported APIs replace the legacy path without changing specified authentication behavior	Verification / Evidence Pass builds, auth tests, compatibility cases, and scans; return commands, results, and criteria mapping	Escalate Return control on role, schema, security-baseline, or behavioral ambiguity	Human gates Approve the plan, pull request, release, and production exposure separately

The contract grants bounded authority—not a blank check.



The deliverable is code plus evidence plus uncertainty

A modernization result is reviewable only when the change arrives with reproducible evidence and an honest account of what remains uncertain.

CODE

- Describe what changed and why each change was necessary

EVIDENCE

- Return exact commands and the resulting build, test, and security outcomes
- Map evidence to every acceptance criterion in the Delegation Contract

UNCERTAINTY

- List unresolved items and assumptions that influenced implementation
- Ask explicit questions where human judgment or additional context is required

Evidence turns generated output into a decision-ready proposal.



The pull request is the governance boundary

The pull request reconnects intent, delegated execution, verification, and accountable human acceptance.

1 Contract + issue

2 Agent session

3 Branch + change

4 Automated checks

5 Context and risk review

6 Required human approvals

7 Separate merge, release, and production gates

- Link the Delegation Contract and issue to the agent session, branch, and resulting change
- Require builds, tests, policy checks, and security scans appropriate to the risk
- Review architectural fit, compatibility, operational impact, and unresolved uncertainty
- Route required security, architecture, and ownership approvals to accountable humans
- Treat merge, release, and production deployment as explicit and potentially separate gates

Autonomous execution does not require autonomous acceptance.



Review the decision, not only the diff

A technically clean diff can still implement the wrong intent, cross a boundary, or create unacceptable operational risk.

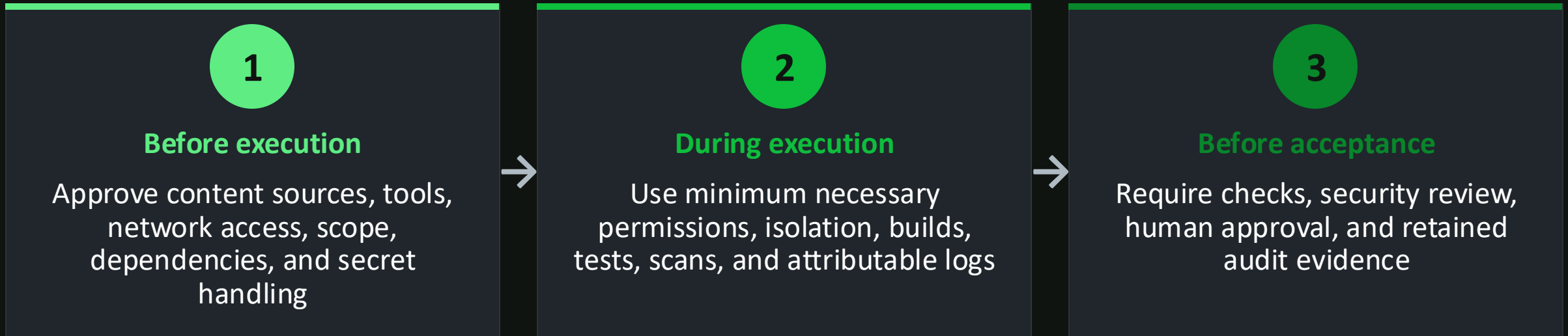
- | | |
|---|--|
| 1 | Intent and scope: Does the change solve the contracted problem without expanding it? |
| 2 | <u>Architecture and compatibility:</u> Does it preserve boundaries, interfaces, and required behavior? |
| 3 | <u>Security:</u> Are trust assumptions, dependencies, data handling, and permissions acceptable? |
| 4 | <u>Verification:</u> Do the tests and evidence support every acceptance claim? |
| 5 | <u>Operability and reversibility:</u> Can teams observe, recover, roll back, and support the change? |

Review whether the change should be accepted—not merely whether it compiles.



Security is a delegation condition

Security determines whether work may be delegated, how it may execute, and what must be true before humans can accept it.



STOP AND ESCALATE

- Stop and escalate on unexpected dependencies, sensitive data, permission gaps, or policy violations

Security belongs in the contract, execution loop, and acceptance gate.



Modernization must be reversible

Every modernization increment needs three things: a bounded change, evidence that it works, and a tested recovery path.

Reversible design

- Keep the change small and compatibility-preserving; use a feature flag or replaceable seam where appropriate
- Define rollback steps and retain the previous authentication path until acceptance is complete
- Use staged rollout, data compatibility, and observable recovery criteria

Evidence and accountable human control

- Capture evidence needed to choose continue, pause, or reverse
- Keep production activation and rollback decisions under accountable human control

If the team cannot explain how to recover, the change is not ready to release.



PR to Learning closes the loop

An accepted pull request becomes organizational learning only when operational signals update future work and repository knowledge.

OPERATIONAL SIGNALS

- Observe telemetry, performance, incidents, support cases, security findings, and user feedback through existing systems

ENCODE LEARNING

- Convert findings into backlog items and stronger or new tests
- Update runbooks, documentation, repository instructions, and reusable skills

REFINE AND DECIDE

- Refine acceptance criteria and the remaining modernization plan
- Let accountable owners decide which signals change priorities, controls, or rollout

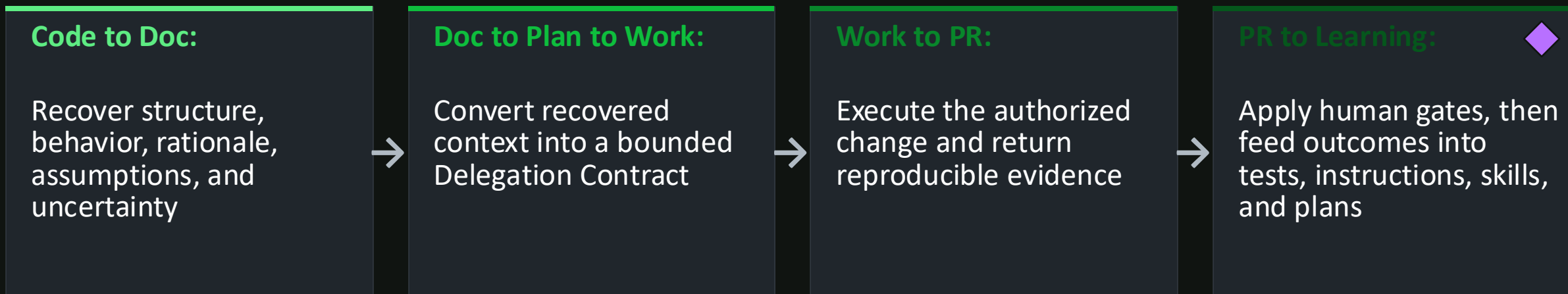
FEEDBACK Accepted operational outcomes → repository context → the next Delegation Contract

The next Delegation Contract should be better because the previous change reached production.



Code to Doc to PR to Learning

This is the collapsed operational view of the five transformations introduced: bounded delegation produces change, explains it, governs acceptance, and improves the next task.



Center: The Delegation Contract preserves intent and boundaries across every stage

↻ **Feedback into future work** human gates approve; evidence and learning return to Code to Doc and the repository

The loop compounds only when evidence and learning return to the repository.



Humans move from the work loop to the decision loop

Agents can perform more investigative and implementation work, while humans concentrate on purpose, trade-offs, risk, and accountable decisions.

AGENT WORK LOOP ↻

- Agents investigate, prepare plans, implement bounded changes, run checks, and collect evidence

HUMAN DECISION LOOP ↻

- Humans set purpose, priority, desired outcomes, and acceptable risk
- Humans decide architecture, compatibility trade-offs, and exceptions
- Humans retain compliance, production, and accountability decisions

GATES RECONNECT THE LOOPS

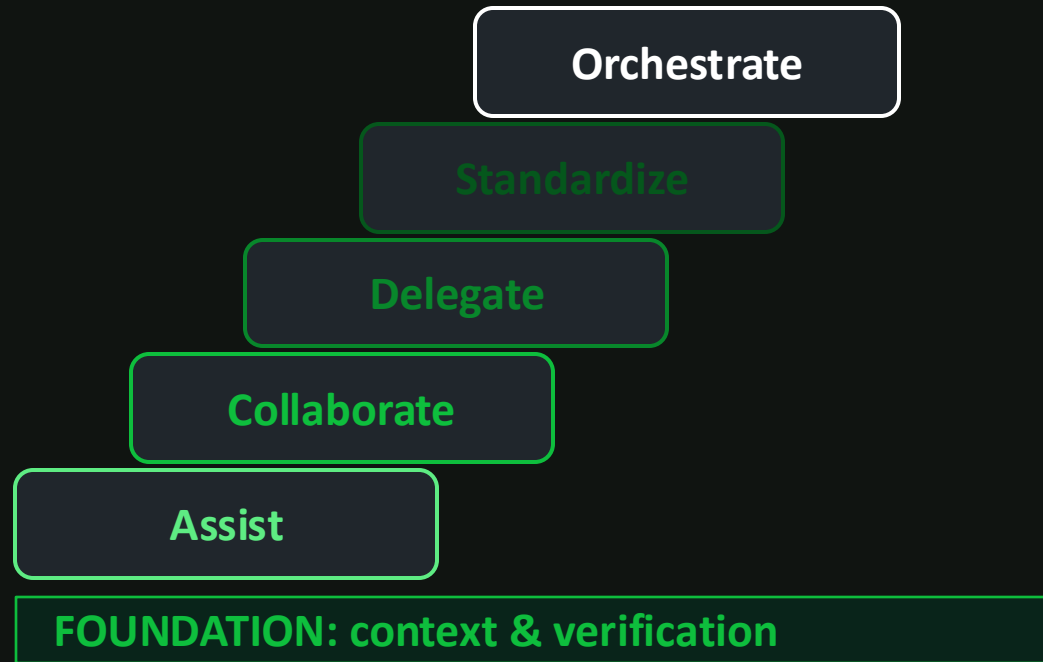
- Declared gates reconnect the execution loop to the decision loop

Agents move the work forward; humans remain accountable for judgment.



Adopt Agentic SDLC as a ladder

This organizational maturity ladder is distinct from the interaction continuum in §2: increase delegation only after repository context and verification mature enough to support the next level safely.



Assist: A human owns every step and uses Copilot for bounded suggestions

Collaborate: Human and agent share investigation, planning, implementation, and checking

Delegate: An agent executes a bounded contract and returns a decision-ready pull request

Standardize: Teams reuse instructions, skills, workflows, evidence formats, and gates

Orchestrate: Multiple bounded tasks run across the portfolio within governance and review capacity

Context and verification mature first; autonomy and orchestration follow.



Tomorrow: write one Delegation Contract

Choose one important, reversible modernization task and write its Delegation Contract before your next sprint planning session.

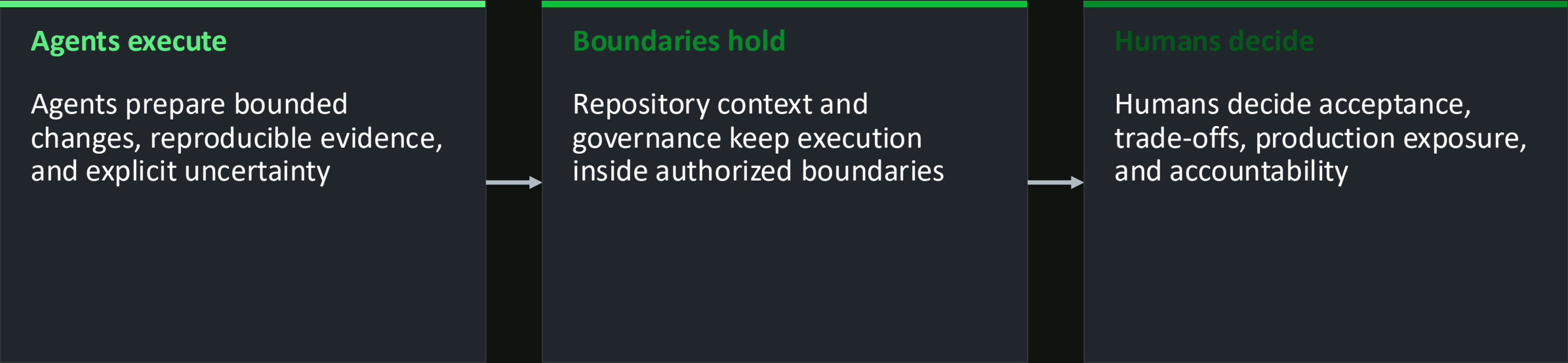
Outcome + Why — the observable result and why it matters	Scope + Out — where work may act and the explicit exclusions	Context — current behavior, repository guidance, decisions, and assumptions	Tools + Constraints — approved capabilities, compatibility, policy, permissions, and budgets
Acceptance — the observable behavior required for done	Verification + Evidence — repeatable checks, commands, results, and criteria mapping	Escalate — stop conditions, ambiguity, and questions to return	Human gates — required decisions before merge, release, or production

Bring the contract—not an open-ended prompt—to sprint planning.



Delegate the work that creates a decision-ready state

GitHub Copilot's next value is not generating more code; it is enabling teams to redesign modernization as bounded, verifiable work that agents execute and humans govern.



Agents do not take the judgment; they prepare the state in which humans can judge.



What is the first modernization task you can delegate?

GitHub Copilot's next value is not generating more code; it is enabling teams to redesign modernization as bounded, verifiable work that agents execute and humans govern.

? What important outcome can be bounded?

? What evidence would make the result decision-ready?

? When must the agent stop and return control?

One important task. One bounded contract. One accountable owner.

